

程式設計與實習-DFS迷宮作業

A1135511詹育綸

- 製作階段: 第三階段
- 作業製作想法、構思

題目敘述: 請讓使用者可以自由選擇迷宮大小(最大80x80), 並且生成一個迷宮, 秀出起點和終點, 其中終點的位置是距離起點最遠的點, 最後印出起點到終點的路徑

1. 迷宮挖洞: 根據題目的要求, 我一開始也想依照講義中的方式進行挖洞, 但因為挖洞的方向須隨機, 不能如同找路徑那樣依照一定順序來進行挖洞, 所以我嘗試了幾個方法。

1) 第一種:

多維陣列 `dir[x座標][y座標][方位]` //方位0123代表北東南西

說明: 運用`rand()`將要拓展道路座標的方位進行隨機選擇, 並將選擇好的方位設為1(已使用), 每次要拓展道路時皆從還未選擇的方向再次進行隨機選擇, 直至迷宮無法再拓展空間。

```
int maze[80][80], dir[80][80][4] = { 0 }; // 0=通道, 1=牆, 2=路徑
bool visited[80][80]; // DFS 記錄用
bool found = false;
```

一開始的變數除了maze之外
額外多一個dir來記錄方向

```
// 利用 DFS 挖迷宮
void generate_maze(int x, int y) {
    maze[x][y] = 0;
    int start_dir = rand() % 4, temp_dir = start_dir;
    int nx = x, ny = y, flag = 0;
    while (flag < 4) {
        nx = x; ny = y; // 每次重設座標

        if (temp_dir == 0) { // 北
            nx = x - 2;
            if (nx > 0 && maze[nx][ny] == 1) {
                dir[nx][ny][2] = 1;
                maze[x - 1][y] = 0;
                generate_maze(nx, ny);
            }
        }
        else if (temp_dir == 1) {
            ny = y + 2;
            if (ny < side_y - 1 && maze[nx][ny] == 1) {
                dir[nx][ny][3] = 1;
                maze[x][y + 1] = 0;
                generate_maze(nx, ny);
            }
        }
    }
}
```

```
    else if (temp_dir == 2) {
        nx = x + 2;
        if (nx < side_x - 1 && maze[nx][ny] == 1) {
            dir[nx][ny][0] = 1;
            maze[x + 1][y] = 0;
            generate_maze(nx, ny);
        }
    }
    else { // 西
        ny = y - 2;
        if (ny > 0 && maze[nx][ny] == 1) {
            dir[nx][ny][1] = 1;
            maze[x][y - 1] = 0;
            generate_maze(nx, ny);
        }
    }

    dir[x][y][temp_dir] = 1;
    temp_dir = (temp_dir + 1) % 4;
    flag++;
}
```

函式前面用`rand()`來從0123擇一為起始方向, 當進到while時, 依照一開始的方向數字開始進行順時針的拓展, 例如: `rand()`隨機出現1(東), 則進到while迴圈, 開始依照2(南)>3(西)>0(北)>1(東), 重複→結束while迴圈

成果顯示:

10 x 20的迷宮挖洞結果

```
#####
#                                     # ##
##### # ##
#                                     # ##
# ##### # # ##
#      # # #
# ##### #####
# #                                     ##
#####
#####
```

2) 第二種:

方向陣列 `dx[4] = { 0, 0, 2, -2 }` `dy[4] = { 2, -2, 0, 0 }`

//xy子集的組合有點像是方向向量 `{ ( 0, 2), ( 0, -2), ( 2, 0), ( -2, 0) }`

說明: 後來想到之前也有寫到一題跟方位有關的CPE題目(UVA-118), 是運用dx,dy子集中index為0123的組合製作成方向向量的形式, 雖然

也可以跟上一種方式一樣用rand()+順時鐘的方式來隨機生成迷宮，但我想說有沒有一種方式可以不用按照「順序」來產生迷宮，因此在詢問ChatGPT下，得知了一個新函數random_shuffle(子集打亂開始位置,子集打亂結束位置)，他可以將我的dx和dy隨機打亂子集順序，但為了避免xy配對到錯誤的index，因此用另一個dirs來進行打亂並套用在dx、dy上。

```
// 分別代表xy座標的(右, 左, 下, 上)
int dx[4] = { 0, 0, 2, -2 };
int dy[4] = { 2, -2, 0, 0 };
```

在最前方新增一個全域變數
來分別代表四個方向向量

```
// 隨機 DFS 挖迷宮
void generate_maze(int x, int y) {
    maze[x][y] = 0;
    vector<int> dirs = { 0, 1, 2, 3 };
    random_shuffle(dirs.begin(), dirs.end()); //隨機打亂dirs中元素順序

    for (int i = 0; i < 4; ++i) {
        int nx = x + dx[dirs[i]];
        int ny = y + dy[dirs[i]];
        if ((nx > 0 && nx < side_x - 1) && (ny > 0 && ny < side_y - 1) && maze[nx][ny] == 1) {
            maze[x + dx[dirs[i]] / 2][y + dy[dirs[i]] / 2] = 0; // 打通中間的牆
            generate_maze(nx, ny);
        }
    }
}
```

裡面的dirs打亂並當成隨機dx、dy的index，此外，由於dx、dy的子及皆為2的倍數，因此在挖跳兩格中間的那格時，可以直接以/2的方式進行index的填寫

成果顯示：

10 x 20的迷宮挖洞結果

也可與上述方式達到相同結果，
但程式碼的部分卻可以縮減許多

```
#####
# #           # #
# # ### ##### # ###
# # # # # # # #
# # # ### # # # #
# # # # # # # #
# ### # ### # ###
# # # # # # #
#####
#####
```

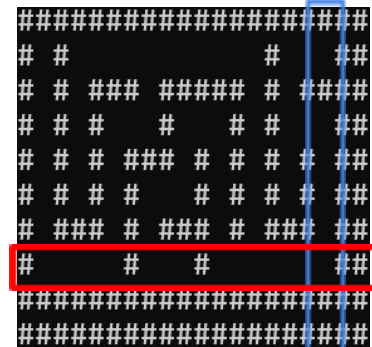
2. 迷宮邊界問題:其實從剛剛的程式碼成果就可看出,似乎只要使用者輸入偶數的迷宮大小的話,邊界就會產生重牆壁,這個問題可從挖洞的方式看出,因為程式碼是從起始位置依序枉死個方向進行兩格兩隔的挖洞處理,因此挖洞的長與寬為 $1+2n$,又為了避免挖洞時凸出牆壁,因此有加設了邊界的判定

```
nx > 0 && nx < side_x - 1) && (ny > 0 && ny < side_y - 1)
```

範例:

在第八行往南進行挖洞時
會因為不滿足`<side_x-1`而停下

在第十九列往東進行挖洞時
同樣會因為不滿足`<side_y-1`而停下



- 1) 第一種:直接不去理會

說明:為了避免影響挖洞的方式而大肆更改程式碼,因此在最一開始使用此方式。

- 優點:程式碼不用更改,最方便的方案
- 缺點:成果顯示會形成雙重牆壁,不僅外觀上顯得很不對稱,且也會讓使用者覺得很沒有必要

- 2) 第二種:強制轉換成奇數的迷宮

說明:若使用者輸入迷宮大小為偶數,則強制將數值大小+or-1(本次作業使用-1的方式進行,避免超出陣列上限),使後續印出迷宮時,直接不顯示雙重牆壁的部分,讓成果畫面更加簡潔

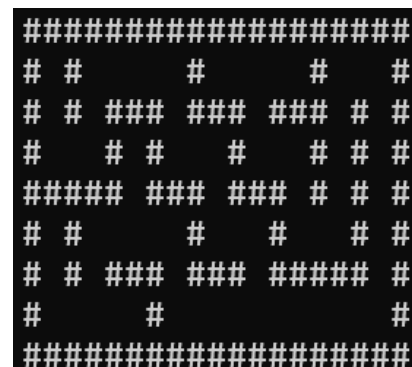
- 優點:程式碼僅加一小部分即可,也避免雙重牆壁的出現
- 缺點:如有使用者細數牆壁的格數,則會顯得該程式詐欺使用者,且與原本預想题目的輸出結果不符

```
//不為基數格,直接改為奇數  
if (side_x % 2 == 0) side_x--;  
if (side_y % 2 == 0) side_y--;
```

新增程式碼在
主程式的輸入範圍後

成果顯示:

10 x 20的迷宮挖洞結果



- 3) 第三種:邊界改為隨機挖一格

說明:當時起初的想法是在其中的幾次挖洞改為一次挖三格,但考量隨機的問題,試了幾次都會導致程式碼出錯,或是直接將迷宮挖出一個大洞。因此,如果只為了將邊界避免有雙重牆壁的問題出現,則只要在雙層牆壁前一格做為判定條件,並且限制僅有道路的相鄰處才可

挖洞，以及洞跟洞之間必須有牆壁(避免找路陷入迴圈)

- 優點:在避免詐欺使用者的情況下將迷宮的邊界顯示的稍微不那麼明顯, 且也解決雙層牆壁的問題
- 缺點:因為邊界挖洞也是機率性挖洞, 所以也有可能出現邊界完全沒挖洞的情況(畢竟機率是一半一半嘛)

```
// 從(1,1)開始挖迷宮
generate_maze(1, 1);
if (side_x % 2 == 0) border_hole_y();
if (side_y % 2 == 0) border_hole_x();
```

新增判定邊界挖洞在生成迷宮之後

```
//邊界隨機挖洞
void border_hole_x() {
    for (int i = 1; i < side_y - 1; i++) {
        int percent = rand() % 10 + 1; //設定一半的機率會挖洞
        if (maze[side_x - 3][i] == 0 && percent > 5 && maze[side_x - 2][i - 1] == 1) {
            maze[side_x - 2][i] = 0;
        }
    }
}

void border_hole_y() {
    for (int i = 1; i < side_x - 1; i++) {
        int percent = rand() % 10 + 1; //設定一半的機率會挖洞
        if (maze[i][side_y - 3] == 0 && percent > 5 && maze[i - 1][side_y - 2] == 1) {
            maze[i][side_y - 2] = 0;
        }
    }
}
```

新增副程式

判定挖洞條件是當random的數值在5以上(數值為1~10)

且長寬分別判定

避免用戶輸入長寬一個為偶數一個為奇數

成果顯示:

10 x 20的迷宮挖洞結果

```
#####
#   #           #   #
### ##### # # ####
#   #           #   #
# ### ### # ##### ##
# #  #  # # #  # ##
# # ### ##### # # ##
#   #           #   #
## ##### # ##### ##
#####
```

3. 設定起始位置:原本就只是將位置設置在(1,1)就好(畢竟挖洞也是從(1,1)開始), 但當時不知道發了什麼神經, 想說如果每次的期使位置皆在不同位置會發生什麼事, 因此就在決定在道路中隨機選擇起點

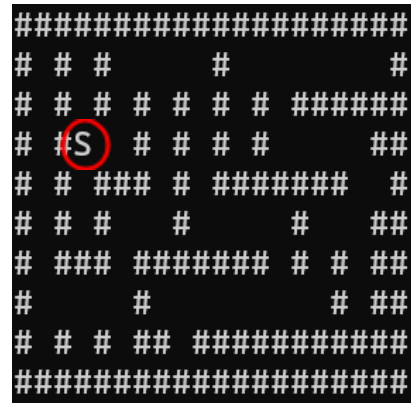
```
// 隨機從通道中選起點
int $_x[6400], $_y[6400], num = 0, num_start = 0;
for (int i = 1; i < side_x - 1; i++) {
    for (int j = 1; j < side_y - 1; j++) {
        if (maze[i][j] == 0) {
            $_x[num] = i;
            $_y[num] = j;
            num++;
        }
    }
}

num_start = rand() % num;
start_x = $_x[num_start];
start_y = $_y[num_start];
```

其實也就是在道路上
隨機選擇一處為起始點

成果顯示：
10 x 20的迷宮挖洞結果

起始位置預設為圖中S的位置



4. 找最遠位置:這個問題在我剛看到這個問題時就在想了,起初想說可以依照起始位置和與DFS找路一樣的方式邊紀錄目前走的路途長度,一邊繼續往下走,結果後續再找DFS迷宮的相關資料時,得知有一方法BFS,因此又嘗試做了一次。

1) 第一種:改編DFS的找路方法

說明:基本上就是用DFS跑過整張地圖,只是額外新增了一個判定長度的條件,當找到的路徑長度超越(先前的)最大值時,設定該點為新的結束位置

```
//DFS找路
void find_way(int x, int y, int depth) {
    if (x<1 || y<1 || x>side_x - 1 || y>side_y - 1) return;
    if (maze[x][y] == 1 || visited_find_way[x][y]) return;
    visited_find_way[x][y] = true;

    if (depth > max_depth) {
        max_depth = depth;
        goal_x = x;
        goal_y = y;
    }

    find_way(x, y + 1, depth + 1); // 右
    find_way(x, y - 1, depth + 1); // 左
    find_way(x + 1, y, depth + 1); // 下
    find_way(x - 1, y, depth + 1); // 上
}
```

新增程式碼

中間的depth為判定目前是否為最長長度的條件判斷

2) 第二種:BFS找路(廣度優先搜尋)

說明:概念有點類似從起始點「蔓延」出去的最遠道路,依序往外擴增判定的道路,並且搭配queue(FIFO)導致在執行的最後一次必定為最遠的格子

```
//從起點BFS找最遠的點
pair<int, int> find_furthest_point(int sx, int sy) {
    queue<pair<int, int>> q;
    bool visited_bfs[80][80] = { false };
    q.push({ sx, sy });
    visited_bfs[sx][sy] = true;

    pair<int, int> furthest = { sx, sy };

    //檢查順序也依序為(右, 左, 下, 上)
    int mx[4] = { 0, 0, 1, -1 };
    int my[4] = { 1, -1, 0, 0 };

    while (!q.empty()) {
        pair<int, int> current = q.front(); q.pop();
        int x = current.first;
        int y = current.second;
        furthest = { x, y }; //假設先前最遠距離

        for (int i = 0; i < 4; ++i) {
            int nx = x + mx[i];
            int ny = y + my[i];
            if (nx >= 0 && nx < side_x && ny >= 0 && ny < side_y && maze[nx][ny] == 0 && !visited_bfs[nx][ny]) {
                visited_bfs[nx][ny] = true;
                q.push({ nx, ny });
            }
        }
    }

    return furthest;
}
```

主要是運用queue的方式

使最後一個執行判定的格子為最遠處

比較偷懶一點，把所有判定條件全寫在最前面Owo

```
// DFS 顯示路徑
void dfs(int x, int y) {
    if (found || x < 0 || y < 0 || x >= side_x || y >= side_y) return;
    if (maze[x][y] == 1 || visited[x][y]) return;

    visited[x][y] = true;
    maze[x][y] = 2;
    print_maze();

    if (x == goal_x && y == goal_y) {
        found = true;
        return;
    }

    dfs(x, y + 1); // 右
    dfs(x, y - 1); // 左
    dfs(x + 1, y); // 下
    dfs(x - 1, y); // 上

    // 將錯誤路徑刪除
    if (found) return;
    maze[x][y] = 0;
}
```

將判定條件全搬去前面
程式碼也比較簡潔

6. 列印道路：這部分也沒什麼好說的，同樣為上課就已知的内容，頂多就將起點與終點的顯示改成特殊字母(之前看到的S和E)，另外就是如果想避免眼睛被閃蝦的話可以用一個ChatGPT推薦的模式，將每次的畫面改為覆蓋上去的方式來避免閃爍，雖然當迷宮過大時仍會不停閃爍。

```
// 迷宮顯示
void print_maze() {
    system("cls");
    for (int i = 0; i < side_x; i++) {
        for (int j = 0; j < side_y; j++) {
            if (i == start_x && j == start_y) cout << "S";
            else if (i == goal_x && j == goal_y) cout << "E";
            else if (maze[i][j] == 1) cout << "#";
            else if (maze[i][j] == 2) cout << "*";
            else cout << " ";
        }
        cout << endl;
    }
    Sleep(50); // 延遲以產生動畫效果
}
```

新增判定起始與終點的符號判定

- GPT推薦的那我也看不懂的程式(因為這是直接生成的我就不直接說明了~)

```
void moveCursorToTop() {
    COORD coord = { 0, 0 };
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
}
```

```
moveCursorToTop();
```

將剛剛print_maze副程式裡
system("cls")改成這個

7. 執行判定與路徑長計算：這部分也不是特別複雜，前者只需在每次呼叫dfs

時，把step_count加一即可；後者一開始想很複雜，想說如果dfs沒找到路的話是否還要扣掉錯誤路徑，但後來想了想只要把最終地圖的*、S、E都算進去就好(其實就是maze[x][y]==2的格子)

```
int step_count = 0, way = 0; 設定全域變數
```

計

```
// DFS 顯示路徑
void dfs(int x, int y) {
    if (found || x < 0 || y < 0 || x >= side_x || y >= side_y) return;
    if (maze[x][y] == 1 || visited[x][y]) return;

    step_count++;
    visited[x][y] = true;
    maze[x][y] = 2;
    print_maze();

    if (x == goal_x && y == goal_y) {
        found = true;
        return;
    }

    dfs(x, y + 1); // 右
    dfs(x, y - 1); // 左
    dfs(x + 1, y); // 下
    dfs(x - 1, y); // 上

    //將錯誤路徑刪除
    if (found) return;
    maze[x][y] = 0;
}
```

算搜尋次數

```
for (int i = 1; i < side_x-1; i++) {
    for (int j = 1; j < side_y; j++) {
        if (maze[i][j] == 2) way++;
    }
}

cout << "搜尋次數：" << step_count - 1 << " 路徑長：" << way << endl;
```

找路徑長與最終輸出

最終成果顯示：
10 x 20的迷宮挖洞結果

```
#####
#  S#  *****  ##
###*###*#####  #
#  ***#***#E#*#  ##
#  ***#***#*#*#  ##
#*****#  ***#*#  ##
#*#####*#  ####
#*****#  #
#####  #  #  #  #  #
#####
完成！
搜尋次數：70 路徑長：47
```

8. 參考資料：

ShengYu Talk - random_shuffle產生不重複的隨機亂數:

<https://reurl.cc/3K9Qv8>

DFS深度優先搜尋: <https://reurl.cc/knggOK>

DFS與BFS: <https://reurl.cc/dQAADk>

NTUCPC Guide - 廣度優先搜尋: <https://reurl.cc/0KRRmA>

ChatGPT: 這肯定是有的, 我需要他解釋一些DFS和BFS的運作原理~

- **加強班 or 助教課的心得**

總之...先感謝您願意開加強班, 我覺得我的程式編寫能力真的有在向上攀升, 從起先只會基礎C語言的我, 現在都已經可以用C++解決大部分問題; 另外就是這也是我第一次是由非課堂老師授課的課程, 其實對我來說蠻新奇的, 但好在上課氣氛也不會到太壓抑或令人想睡覺, 我覺得很讚。最後就是也感謝您每次都會盡可能地解決我提出的各種問題, 其實我不是那種敢在課堂直接發問的那種人, 因此這種私下問問題的感覺讓我很安逸(?總之這堂課讓我獲益良多, 也期望你能得到優秀助教的殊榮~(我其實忘記那個名字了)

阿對了, 雖然保榮快退休了, 但助教課好像會讓大家上他的課時, 反而變得沒那麼認真, 呃...幫保榮QQ